IEEE Open Journal of
Circuits and Systems

Received 30 December 2024; revised 6 April 2025; accepted 4 July 2025. Date of publication 22 July 2025; date of current version 22 August 2025.

Digital Object Identifier 10.1109/OJCAS.2025.3591136

Highly-Efficient Hardware Architecture for ML-KEM PQC Standard

HAESUNG JUNG (Graduate Student Member, IEEE), QUANG DANG TRUONG (Graduate Student Member, IEEE), AND HANHO LEE (Senior Member, IEEE)

Department of Electrical and Computer Engineering, Inha University, Incheon 22212, South Korea

This article was recommended by Guest Editor B. Wang.

CORRESPONDING AUTHOR: H. LEE (e-mail: hhlee@inha.ac.kr)

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP)-Information Technology Research Center (ITRC) Grant funded by the Korea Government (Ministry of Science and ICT, MSIT) under Grant RS-2021-II212052, and in part by the Commercialization Promotion Agency for Research and Development Outcomes (COMPA) funded by MSIT under Grant RS-2025-02219156.

(Haesung Jung and Quang Dang Truong are co-first authors.)

ABSTRACT The advent of quantum computers, with their immense computational potential, poses significant threats to traditional cryptographic systems. In response, NIST announced the quantum-resistant Module Lattice-based Key Encapsulation Mechanism (ML-KEM) standard in 2024. This paper presents an efficient hardware architecture for the ML-KEM scheme, capable of supporting all algorithms and flexibly adapting to different security levels. The proposed design achieves a balance between high performance and low hardware resource consumption, making it suitable for deployment across various FPGA platforms. Key innovations include the Unified Polynomial Arithmetic Module (UniPAM), capable of handling all polynomial arithmetic operations, and an optimized hash module for the SHA-3 variants integral to ML-KEM. Additionally, the design introduces an efficient timing diagram and conflict-free memory management strategy, enabling seamless parallelism and reducing execution time while minimizing hardware resource consumption. Furthermore, the implementation incorporates several methods to effectively mitigate side-channel attacks, a common concern in hardware-based cryptosystem deployments. The proposed architecture is validated through implementation on an Artix-7 FPGA and Synopsys 14nm ASIC technology. Compared to state-of-the-art designs, our approach demonstrates superior performance while maintaining comparable hardware resource efficiency. Specifically, the hardware implementation on the Xilinx Artix-7 utilizes 12k LUTs, 6.9k FFs, 4 DSPs, and 9 BRAMs at clock frequency of 220 MHz.

INDEX TERMS Key encapsulation mechanism, post-quantum cryptography, number theoretic transform, Secure Hash Algorithm 3, CRYSTALS-Kyber.

I. INTRODUCTION

IN THE Internet era, the security requirements for conventional public-key cryptographic algorithms such as RSA and Elliptic Curve Cryptography (ECC) face serious threats from the advent of large-scale quantum computers. Companies and organizations worldwide, including Google, IBM, and Intel, are actively investing in quantum computer development, with commercialization expected within the next decade. Quantum computers leverage algorithms such as Shor's and Grover's to efficiently solve problems like integer factorization and discrete logarithms, which form the foundation of many existing cryptographic schemes [1]. To address these threats, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardization project in 2016. This project aims to establish cryptographic algorithms secure against quantum attacks while remaining operable on classical computers. After extensive evaluation, CRYSTALS-Kyber, a lattice-based scheme, was selected as the primary algorithm for the Module Lattice-based Key Encapsulation Mechanism

(ML-KEM) and published in August 2024 as FIPS 203 [2].

As a lattice-based cryptosystem, ML-KEM relies on the hardness of the Module Learning With Errors (MLWE) problem [2], which involves computationally intensive sampling and polynomial matrix-vector multiplications. To mitigate the complexity of polynomial multiplications, ML-KEM employs Number Theoretic Transform (NTT), which reduces computational complexity from $O(n^2)$ to $O(n\log n)$ from traditional school-book method. However, this approach necessitates efficient implementation of NTT and its inverse transformation (INTT) on application platforms. These requirements call for affordable platforms that balance high performance with low cost while mitigating side-channel and fault attacks, which are prevalent in software implementations. In this context, hardware implementations–particularly FPGA technology–demonstrate superior performance due to their flexibility in configuration using basic hardware elements. FPGAs efficiently integrate both logic and arithmetic operations, making them ideal for implementing Secure Hash Algorithm 3 (SHA-3) [3], [4] and NTT-based arithmetic operations required in ML-KEM. Additionally, ASICs present an effective solution for designing specialized chips for low-cost applications, enabling widespread deployment worldwide.

With the potential to become the new NIST standard, numerous hardware implementations for CRYSTALS-Kyber have been proposed in recent years [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], and the most notable ones are summarized here. In [16], a high-performance Kyber implementation is proposed using a radix-2 Multipath Delay Commutator (MDC) NTT module, which significantly boosts the computation process's execution time. Other state-of-the-art (SOTA) studies focus on optimizing Kyber implementations by reducing hardware resource consumption to minimize the Area-Time Product (ATP) metric. The works in [17] and [18] propose highly compact architectures by utilizing double Butterfly Units (BUs) in the arithmetic module. This approach is considered optimal for efficiency based on the ATP metric, but it suffers from low performance, which often becomes a bottleneck in NTT-based operations. To improve performance, Guo and Li in [19] and [20] proposed split-radix and mixed-radix methods, achieving a 30–40% performance improvement over [17]. However, the descriptions of these methods and their hardware architectures are sparse, providing insufficient information for constructing these architectures in hardware. Furthermore, these papers lack comprehensive timing diagrams for the entire architecture across the three ML-KEM main algorithms. Such diagrams are crucial to ensuring efficient implementation by optimizing the usage of hardware submodules.

While Kyber serves as the foundational algorithm, the ML-KEM standard introduces modifications and differs from Kyber's final submission to the NIST competition. These differences underscore the necessity for updated implementations aligned with the published standard (FIPS 203) to guarantee the security and performance of the scheme. However, most existing hardware implementations are based on the now-outdated CRYSTALS-Kyber scheme, which lacks compliance with the latest standard. This gap underscores the urgency of transitioning to designs aligned with ML-KEM to guarantee robustness against emerging quantum threats. Moreover, except for the recent works in [21], [22], no prior studies have claimed support for all three main ML-KEM algorithms at the three defined security levels. Some implementations, such as those in [14], [15], [16], only target independent security levels, while others, including [16] and [17], are designed for static client/server architectures, which do not allow role interchangeability during the key exchange process. These limitations hinder practical applicability and necessitate cumbersome procedures for end-users. This calls for a configurable architecture capable of executing all ML-KEM protocol tasks and dynamically switching security levels during runtime to meet diverse application needs.

In addition to the imperative for an optimized hardware design of ML-KEM that achieves high performance with competitive resource utilization, ensuring the security of Key Encapsulation Mechanisms (KEMs) in practical applications is crucial. The hardware implementation must meet specific qualifications and critical conditions outlined in the recommended literature [24]. Previous studies have identified vulnerabilities in the Kyber algorithm to side-channel attacks (SCAs), particularly the timing attacks, as detailed in [25]. Additionally, side-channel leakage during Barrett reduction has been observed [26], along with susceptibility to electromagnetic (EM) and power side-channel attacks [27], [28]. These issues have often been overlooked in prior hardware implementations of Kyber. Although ML-KEM has been enhanced from Kyber to improve security, it remains crucial to consider these physical attacks when designing secure implementations. Addressing these concerns is essential to mitigate risks during key exchange in KEM applications.

In this paper, we propose an efficient implementation for ML-KEM on hardware platforms, specifically the Artix-7 FPGA family and Synopsys 14nm Educational Design Kit (EDK) ASIC technology. The proposed configurable architecture supports all ML-KEM algorithms and enables real-time switching among the 3 security levels, addressing limitations in flexibility and functionality of prior works. Moreover, we present detailed hardware architecture specifically tailored for polynomial arithmetic operations (PAOs) and hash functions. We also propose appropriate timing diagram that ensures seamless integration of submodules, achieving the desired performance while maintaining affordable hardware resource consumption across a wide range of platforms. This timing diagram is designed to ensure constant-time execution, enhancing resistance to timing attacks. Furthermore, it is structured to handle multiple parallel tasks, thereby mitigating vulnerabilities to power

Algorithm 1: ML-KEM.KeyGen() [2]

Input: randomness $d \in \mathbb{B}^{32}$, $z \in \mathbb{B}^{32}$
Output: ek $\in \mathbb{B}^{384k+32}$, dk $\in \mathbb{B}^{768k+96}$

$(\rho, \delta) \leftarrow \mathrm{G}(d \parallel k)$
$\hat{A} \in (\mathbb{Z}_q^{256})^{k \times k} \leftarrow \mathrm{XOF}(\rho)$
$\mathbf{s}, \mathbf{e} \in (\mathbb{Z}_q^{256})^k \leftarrow \mathrm{PRF}_{\eta_1}(\delta)$
$\hat{t} \leftarrow \hat{A} \circ \mathrm{NTT}(\mathbf{s}) + \mathrm{NTT}(\mathbf{e})$
ek $\leftarrow (\hat{\mathbf{t}} \parallel \rho)$
dk $\leftarrow (\hat{\mathbf{s}} \parallel$ ek $\parallel \mathrm{H}($ek$) \parallel z)$
return (ek,dk)

Algorithm 2: K-PKE.Encrypt($\mathrm{ek}_{\mathrm{PKE}}$, m, r) [2]

Input: encryption key $\mathrm{ek}_{\mathrm{PKE}} \in \mathbb{B}^{384k+32}$, message $m \in \mathbb{B}^{32}$, encryption randomness $r \in \mathbb{B}^{32}$
Output: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$

$\hat{A} \in \mathbb{Z}_q^{k \times k} \leftarrow \mathrm{XOF}(\rho)$
$\mathbf{y} \in (\mathbb{Z}_q^{256})^k \leftarrow \mathrm{PRF}_{\eta_1}(r)$
$\mathbf{e_1} \in (\mathbb{Z}_q^{256})^k \leftarrow \mathrm{PRF}_{\eta_2}(r)$
$e_2 \in (\mathbb{Z}_q^{256}) \leftarrow \mathrm{PRF}_{\eta_2}(r)$
$\mathbf{u} \leftarrow \mathrm{NTT}^{-1}(\hat{A}^\top \circ \mathrm{NTT}(\mathbf{y})) + \mathbf{e_1}$
$\mu \leftarrow \mathrm{Decompress}_1(\mathrm{ByteDecode}_1(m))$
$v \leftarrow \mathrm{NTT}^{-1}(\hat{t}^\top \circ \hat{y}) + e_2 + \mu$
$c_1 \leftarrow \mathrm{ByteEncode}_{d_u}(\mathrm{Compress}_{d_u}(\mathbf{u}))$
$c_2 \leftarrow \mathrm{ByteEncode}_{d_v}(\mathrm{Compress}_{d_v}(v))$
return $c \leftarrow (c_1 \parallel c_2)$

Algorithm 3: ML-KEM.Encaps(ek) [2]

Input: encapsulation key ek $\in \mathbb{B}^{384k+32}$
randomness $m \in \mathbb{B}^{32}$
Output: shared secret key $K \in \mathbb{B}^{32}$
ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$

$(K, r) \leftarrow \mathrm{G}(m \parallel \mathrm{H}(ek))$
$c \leftarrow$ K-PKE.Encrypt(ek,m,r)
return (K, c)

Algorithm 4: ML-KEM.Decaps(c,dk) [2]

Validated input: dk $\in \mathbb{B}^{768k+96}$, $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output: shared key $K \in \mathbb{B}^{32}$

$(\hat{s} \parallel \hat{t} \parallel \rho \parallel h \parallel z) \leftarrow$ dk
$\mathbf{u}' \leftarrow \mathrm{Decompress}_{d_u}(\mathrm{ByteDecode}_{d_u}(c_1))$
$v' \leftarrow \mathrm{Decompress}_{d_v}(\mathrm{ByteDecode}_{d_v}(c_2))$
$w \leftarrow v' - \mathrm{NTT}^{-1}(\hat{\mathbf{s}}^\top \circ \mathrm{NTT}(\mathbf{u}'))$
$m' \leftarrow \mathrm{ByteEncode}_1(\mathrm{Compress}_1(w))$
$(K', r') \leftarrow \mathrm{G}(m' \parallel h)$
$\bar{K} \leftarrow \mathrm{J}(z \parallel c)$
$c' \leftarrow$ K-PKE.Encrypt(ek, m', r')
if $(c \neq c')$ **then** $K' \leftarrow \bar{K}$
return K'

analysis attacks. Our primary contributions are summarized as follows:

1) We propose an efficient ML-KEM hardware architecture capable of supporting all three security levels and core functionalities: Key Generation, Encapsulation, and Decapsulation within a single implementation. This design enhances convenience by avoiding multiple implementations for client and server protocols and facilitating seamless role and security level exchanges compared to prior works.
2) The two most time-critical submodules in ML-KEM protocol are detailed and optimized. A Unified Polynomial Arithmetic Module (UniPAM) has been introduced, capable of performing all arithmetic operations required by ML-KEM, including NTT, INTT and other functionalities. Additionally, an efficient hash module has been developed to support the four variant hashing functions of SHA-3 utilized in this scheme. These optimizations, along with meticulously synchronized timing diagrams, significantly improve execution speed and resource efficiency.
3) To verify the effectiveness of our proposal, we implemented ML-KEM architecture on the Artix-7 FPGA platform and Synopsys 14nm EDK ASIC technology. The implementation results demonstrate that our design requires only 12k LUTs, 6.9k FFs, 4 DSPs, and 9 BRAMs on the FPGA and occupies 111 nm^2 on ASIC, while providing a fully configurable ML-KEM architecture for all algorithms and security level.

The remainder of this paper is structured as follows: Section II introduces the foundational concepts and algorithms underlying ML-KEM. Section III elaborates on the proposed hardware architecture, including its submodules and the optimized timing diagram tailored for the three primary algorithms. The performance and efficiency of the proposed design are analyzed in Section IV, where implementation results are presented and compared with SOTA works. Finally, the key contributions and conclusions of this study are summarized in Section V.

II. PRELIMINARY

A. MODULE LATTICE-BASED KEY ENCAPSULATION MECHANISM

ML-KEM is a key encapsulation mechanism derived from the CRYSTALS-Kyber algorithm, designed to establish a shared secret key between two communicating parties. This shared secret key can subsequently be used for symmetric-key cryptography. The security of ML-KEM is rooted in the presumed hardness of the Module Learning With Errors (MLWE) problem, which itself is based on the computational challenges posed by certain problems in module lattices [2]. The construction of ML-KEM involves two primary steps. First, the MLWE problem is employed to develop a public-key encryption (PKE) scheme. This PKE scheme is then converted into a key encapsulation mechanism using the Fujisaki-Okamoto transform. Unlike the original Kyber specification, the PKE scheme in ML-KEM is not used as a standalone system but rather as a set of subroutines within the core algorithms of ML-KEM.

Algorithm 5: Mixed-Radix-4/2 NTT Algorithm

Input: $a = (a[0], a[1],..., a[n\text{-}1]) \in \mathbb{Z}_q$.
Output: $\hat{a} = \mathrm{NTT_n}(a)$
$i = 4^p$; $t = 0$; ω_1^4 is 4^{th} primitive roots of unity.
for $(p = 3;\ p > 0;\ p{-}{-})$ **do**
  for $(k = 0;\ k < N/4i;\ k{+}{+})$ **do**
    $\omega_1, \omega_2, \omega_3 \leftarrow$ R4NTT_ROM[$t{+}{+}$];
    for $(j = 0;\ j < i;\ j{+}{+})$ **do**
      $m = 4ki + j$;
      $t_0 = (a[m] + a[m+2i].\omega_2) \mod q$
      $t_1 = (a[m] - a[m+2i].\omega_2) \mod q$
      $t_2 = (a[m+i].\omega_1 + a[m+3i].\omega_3) \mod q$
      $t_3 = (a[m+i].\omega_1 - a[m+3i].\omega_3) \mod q$
      $a[m] = (t_0 + t_2) \mod q$
      $a[m+i] = (t_0 - t_2) \mod q$
      $a[m+2i] = (t_1 + t_3.\omega_4^1) \mod q$
      $a[m+3i] = (t_1 - t_3.\omega_4^1) \mod q$
for $(j = 0;\ j < N;\ j{=}j{+}4)$ **do**
  $\omega \leftarrow \omega$_ROM[$j/4$];
  $t_0 = a[j+2].\omega \mod q$; $t_1 = a[j+3].\omega \mod q$
  $a[j] = (a[j] + t_0) \mod q$
  $a[j+2] = (a[j] - t_0) \mod q$
  $a[j+1] = (a[j+1] + t_1) \mod q$
  $a[j+3] = (a[j+1] - t_1) \mod q$

Algorithm 6: Mixed-Radix-4/2 INTT Algorithm

Input: $\hat{a} = (\hat{a}[0], \hat{a}[1],..., \hat{a}[n\text{-}1]) \in \mathbb{Z}_q$
Output: $a = \mathrm{INTT_n}(\hat{a})$
for $(j = 0;\ j < N;\ j{=}j{+}4)$ **do**
  $\omega^{-1} \leftarrow \omega^{-1}$_ROM[$j/4$];
  $t_0 = a[j] \mod q$; $t_1 = a[j+1] \mod q$
  $a[j] = (a[j+2] + t_0)/2 \mod q$
  $a[j+2] = (t_0 - a[j+2]).\omega^{-1} \mod q$
  $a[j+1] = (a[j+3] + t_1)/2 \mod q$
  $a[j+3] = (t_1 - a[j+3]).\omega^{-1} \mod q$
$i = 4^p$; $t = 0$;
for $(p = 1;\ p < 4;\ p{+}{+})$ **do**
  for $(k = 0;\ k < N/4i;\ k{+}{+})$ **do**
    $\omega_1^{-1}, \omega_2^{-1}, \omega_3^{-1} \leftarrow$ R4INTT_ROM[$t{+}{+}$];
    for $(j = 0;\ j < i;\ j{+}{+})$ **do**
      $m = 4ki + j$;
      $t_0 = (a[m] + a[m+i])/2 \mod q$
      $t_1 = (a[m] - a[m+i]).\omega_4^1 \mod q$
      $t_2 = (a[m+2i] + a[m+3i])/2 \mod q$
      $t_3 = (a[m+2i] - a[m+3i]) \mod q$
      $a[m] = (t_0 + t_2)/2 \mod q$
      $a[m+i] = (t_1 + t_3).\omega_1^{-1} \mod q$
      $a[m+2i] = (t_0 - t_2).\omega_2^{-1} \mod q$
      $a[m+3i] = (t_1 - t_3).\omega_3^{-1} \mod q$

ML-KEM comprises three core algorithms and a set of parameters corresponding to its three security levels. The algorithms are Key Generation, Encapsulation, and Decapsulation, denoted as KeyGen, Encaps, and Decaps, respectively. The parameter sets for the three security levels are detailed in the official specification [2], and the three main algorithms are summarized in Algorithms 1, 3, and 4. Specifically, the KeyGen algorithm generates a public encapsulation key and a private decapsulation key. With the encapsulation key, the first party executes the Encaps algorithm to produce a copy K of the shared secret key, along with an associated ciphertext. The second party completes the process by running the Decaps algorithm using the decapsulation key and the ciphertext to recover the shared secret key copy K'.

B. NUMBER THEORETIC TRANSFORM (NTT)

The NTT is used to reduce the complexity and improve the efficiency of multiplication in the ring $R_q = \mathbb{Z}_q/(x^n + 1)$, where modulus $q = 3329$, $n = 256$ and field $\mathbb{Z}_q$ contains 128 primitive n-th roots of unity, all the general algorithms, including the PAOs, NTT, INTT and compression algorithms of ML-KEM, are thoroughly detailed in its specification [2]. Specifically, we denote the coordinate-wise multiplication (CWM) algorithm in ML-KEM as: $\hat{h}_{2i} + \hat{h}_{2i+1}X = (\hat{f}_{2i} + \hat{f}_{2i+1}X) \cdot (\hat{g}_{2i} + \hat{g}_{2i+1}X) \mod (X^2 - \zeta^{2br(i)+1})$, which can be transformed using Karatsuba algorithm into 4 multiplications, as quoted in the formula given in [17]:

$$\begin{aligned} \hat{h}_{2i} &= \hat{f}_{2i}\hat{g}_{2i} + \hat{f}_{2i+1}\hat{g}_{2i+1} \cdot \zeta^{2br(i)+1} \\ \hat{h}_{2i+1} &= \left(\hat{f}_{2i} + \hat{f}_{2i+1}\right)\left(\hat{g}_{2i} + \hat{g}_{2i+1}\right) - \hat{f}_{2i}\hat{g}_{2i} - \hat{f}_{2i+1}\hat{g}_{2i+1} \end{aligned} \quad (1)$$

Radix-2/radix-4 NTT multiplication methods are commonly used in hardware implementations of PQC schemes. Based on radix-2/radix-4 NTT algorithms introduced in [29], we propose mixed-radix-4/2 NTT and INTT for ML-KEM, as shown in Algorithms 5 and 6, respectively. Since $\mathbb{Z}_q$ contains n-th primitive roots of unity, the last stage of NTT and first stage of INTT are handled using radix-2, while the remaining three stages use radix-4 method. Following the approach from [30] to eliminate the multiplication of resulting coefficients with $n^{-1} \mod q$ after INTT, the twiddle factors (TFs) in INTT have been pre-calculated to incorporate the factors 2^{-1} or 2^{-2}. All TFs used for NTT and INTT are stored in corresponding ROM positions.

C. SECURE HASH ALGORITHM 3 (SHA-3)

SHA-3 is the latest secure hash algorithm released by NIST in 2015, based on the Keccak algorithm [3]. In ML-KEM, two SHA-3 hash functions, SHA3-256 and SHA3-512, as well as two extendable-output functions (XOFs), SHAKE128 and SHAKE256, are utilized for various purposes, including pseudo-random number generation, key derivation, and hash functions. These functions share the same underlying sponge construction, where data is absorbed into the sponge and then squeezed out. In the absorbing phase, input message blocks are XORed into a subset of the internal state, and the entire state is transformed using the permutation function known as *Keccak-p*, which is the core function used in both the absorbing and squeezing phases of the sponge construction. While *Keccak-p* remains the same across all four instances of SHA-3 and SHAKE, the key differences

lie in the parameters, such as the bit-rate r and capacity c [3]. Notably, SHA-3 restricts the output length d in range of bit-rate r, while SHAKE can generate as many bits as requested, making it an extendable-output function ideal for applications such as sampling generation. The Keccak algorithm primarily involves logic and mapping operations, making it particularly well-suited for implementation on configurable hardware platforms, such as FPGAs and ASICs.

III. HIGH-LEVEL CONFIGURABLE ML-KEM ARCHITECTURE

To achieve a highly efficient implementation of ML-KEM on the FPGA SoC platform, the proposed high-level configurable architecture is designed to support all core algorithms of ML-KEM across the three security levels defined in its specification [2]. This design also encompasses the processes of packing and transmitting ciphertext and key pairs, enabling seamless interoperability with both software and hardware platforms. This section introduces the overall high-level architecture, detailing its optimized modules and the strategies employed to synchronize their operations within the entire system. This design minimizes execution time by maximizing parallelism among submodules whenever possible. Alongside the efficient hardware implementation, timing schedules (illustrated in Figs. 2–4) are proposed to optimize the overall performance, ensuring low-latency operation. These schedules leverage the functionality of the most critical and time-intensive submodules: UniPAM and the specialized hash module tailored for ML-KEM.

A. OVERALL HARDWARE ARCHITECTURE AND OPERATION SCHEDULING

The high-level architecture of our hardware design is shown in Fig. 1. As a configurable architecture that implements main algorithms of ML-KEM, this figure depicts components utilized and their respective data flows. A brief description of these modules and their functions is explained below:

Control unit: Operating as finite-state machine (FSM), the control unit plays critical role in managing the various modules within the architecture. It coordinates their operations based on the required functionalities, ensuring the correct sequence of actions to execute all primary ML-KEM operations. The FSM is designed to orchestrate the submodules according to the dataflow outlined in the operation schedules of the three main algorithms: Keygen, Encaps, and Decaps, as shown in Figs. 2–4. The control unit uses 2 control signals, 'mode' and 'sec_lvl,' to configure the algorithm and security level for operation. These signals dynamically adjust parameters to comply with the values specified in the ML-KEM specification, corresponding to selected security level.

Polynomial memory: This module consists of five memory banks designed to store intermediate data during the execution of algorithms. Specifically, the first memory bank (RAM0) is configured with 64-bit data width to store the seeds (z, ρ, δ, $\bar{K}$, r, $\bar{K}'$, r') required for initializing the

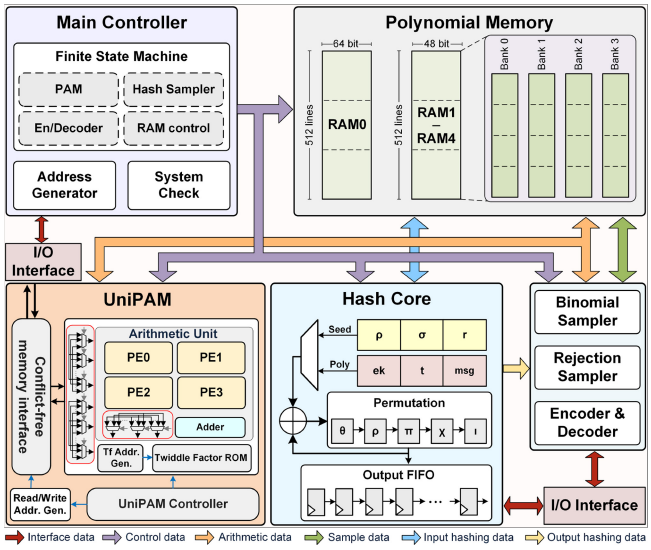

FIGURE 1. High-level configurable ML-KEM architecture.

symmetric primitives used in rejection sampling or central binomial distribution. The remaining memory banks (RAM1 to RAM4) are organized into 4 independent bank columns, each capable of storing up to 512×12-bit data, aligning with the coefficient size of variables in ML-KEM scheme.

Unified polynomial arithmetic module (UniPAM): used for all arithmetic operations in ML-KEM, including NTT, INTT, point-wise multiplications, additions, subtractions and compressions between polynomial matrices and vectors.

Hash module: This configurable module functions as either a hash function or XOFs [3]. It supports switching between four instances of the FIPS 202 standard, which differ in parameters but share the same permutation core: SHA3-256, SHA3-512, SHAKE128, and SHAKE256. These instances are used to generate hash values for messages, random seeds, and pseudorandom data, which are essential for polynomial sampling in the ML-KEM scheme.

Encoder/decoder: To ensure seamless interfacing with other hardware or software platforms, which typically operate with a bandwidth of 2^n, this implementation incorporates encoder and decoder units. The encoder efficiently packs polynomial vectors into byte strings, where various sampled polynomial vectors and seeds, along with their respective bit widths, are compactly packed into fixed 8-byte strings (64-bit bandwidth). The decoder subsequently recovers the original vectors from these byte strings, preparing them for subsequent computational processes.

Operation scheduling: Depending on the algorithm being executed, certain processing units are configured to operate in parallel to minimize overall latency, provided no data flow conflicts arise. As highlighted in algorithms 1–4, the pseudo-random sampling generation and computation phases are the most time-intensive components of the ML-KEM scheme. To mitigate this, this design prioritizes these phases and reduces their execution time by overlapping them wherever feasible, as illustrated in the operation scheduling diagrams

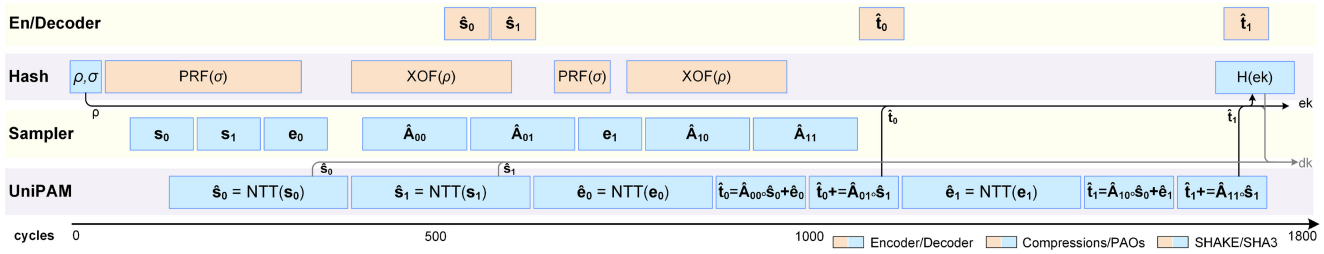

FIGURE 2. Timing schedule for Keygen at security Level 1.

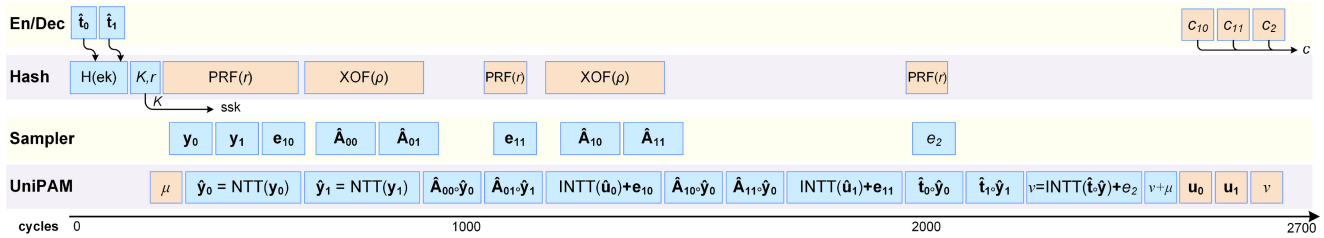

FIGURE 3. Timing schedule for Encaps at security Level 1.

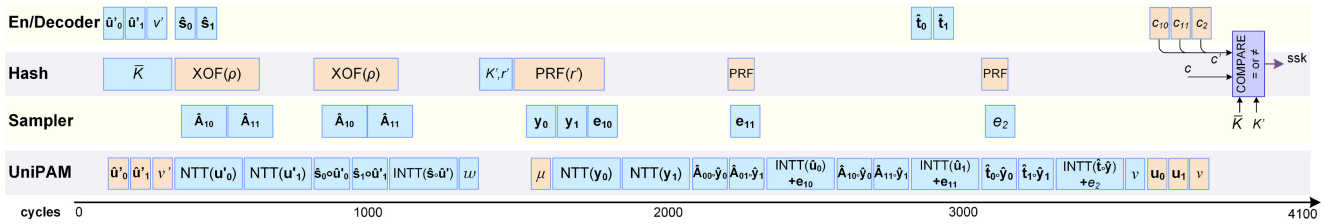

FIGURE 4. Timing schedule for Decaps at security Level 1.

in Figures 2–4. The computational process, handled by the UniPAM, dominates the overall execution time. To optimize this, the sampling and other processes are integrated and hidden within the computational process, effectively reducing unnecessary idle time. This approach ensures that the sampling time is overlapped with computation, improving the execution time of the entire architecture. It is evident that increasing the speed of UniPAM leads to faster execution times. However, this improvement introduces additional architectural complexity and increases hardware resource requirements. After careful evaluation, a UniPAM design capable of handling four coefficients per cycle was selected. This configuration strikes a balance between computation and sampling speeds, resulting in a more compact architecture that avoids data flow conflicts while maintaining high performance. By optimizing the sequencing of tasks and ensuring conflict-free interconnections between its submodules, this design enables continuous execution in a pipelined manner, effectively reducing idle time and significantly boosting overall performance.

To counter SCAs aimed at recovering secret key, as noted in [28], we designed our timing schedule to execute multiple tasks in parallel, especially those involving a secret vector $\mathbf{s}$. As shown in Fig. 2, we integrate NTT operation concurrently with the generation of vector $\mathbf{s}$, and execute public matrix $\hat{A}$ generation during encoding and packing $\mathbf{s}$. This parallel execution of multiple complex tasks obscures EM and power traces, enhancing security. A similar strategy is employed during decapsulation (Fig. 4): decoding and computations involving $\mathbf{s}$ are performed alongside the generation of $\hat{A}$, effectively masking potential leakage from $\mathbf{s}$ and bolstering defenses against EM and power analysis attacks. Moreover, to address timing attack vulnerabilities associated with the variable-time sampling of $\hat{A}$ due to rejection conditions, our timing schedule ensures constant-time execution, mitigating risks highlighted in [25]. Additionally, in line with secure KEMs usage guidelines from [24], shared and decapsulation keys, as well as the input message, are immediately packaged and transmitted post-generation, avoiding memory retention. Memory locations previously holding secret vector $\mathbf{s}$ are promptly overwritten with non-sensitive data after computations, preventing potential leakage through memory analysis.

B. UNIFIED POLYNOMIAL ARITHMETIC MODULE

Fig. 1 shows the proposed UniPAM architecture, comprising three main components: Arithmetic Unit (AU), Control Unit (CU), and Conflict-Free Memory Interface (CMI), which connects the UniPAM to the Polynomial Memory (PM) in the top module. Leveraging a reconfigurable mixed-radix-4/2 butterfly unit, the UniPAM efficiently performs all polynomial arithmetic operations required in ML-KEM.

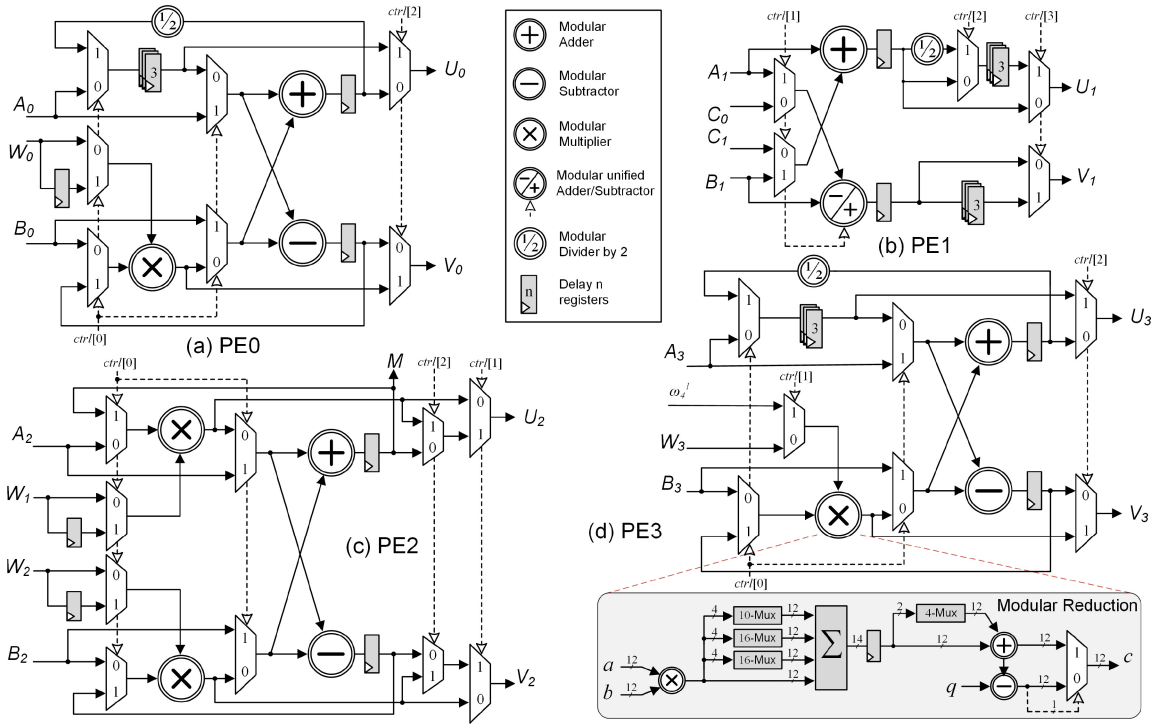

FIGURE 5. Proposed architectures for reconfigurable mixed-radix-4/2 Butterfly unit.

These operations include addition, subtraction, and CWM. Furthermore, by exploiting the rounding characteristics inherent in compression and decompression (Co/Deco) functionalities, these functions are integrated into the UniPAM, thereby reducing unnecessary hardware utilization.

1) MIXED-RADIX RECONFIGURABLE BUTTERFLY UNIT

The reconfigurable mixed-radix-4/2 BU is composed of 4 flexibly reconfigurable processing elements (PEs), as shown in Fig. 5. Based on the requirements of PAOs in ML-KEM, this unit supports 7 operating modes: NTT, INTT, CWM, ADD, SUB, compression and decompression. To enable flexible switching between operating modes, we use 2-Muxs, which are controlled by 4 control signals (*ctrl*[3:0]). These control signals are set as '1010' for NTT mode, '1111' for INTT, '1000' for CWM, '0011' for ADD/SUB, '1100' and '0100' for Co/Deco mode. Let X_i, Y_i and Z_i, where $i = \{0, 1, 2, 3\}$, represent the group of 4 coefficients as inputs and outputs of the AU. Here, X_i and Y_i represent inputs from two polynomials for PAOs, while Z_i represents the output. Depending on the selected mode, inputs and outputs of each PE switch to meet the requirements of each operation, as presented in Table 1, which shows connections inside the AU interconnector in different operating modes, indicated by $mode_{AU}$. The pins named in the gray cells on the left are connected to different pins according to operating mode $mode_{AU}$ on the right. Additionally, to further reduce execution time, an extra modular adder is incorporated. This unit seamlessly transitions from the CWM mode to a multiply-accumulate (MAC) mode, which is commonly utilized for matrix and vector multiplications.

Following the proposed algorithms for mixed-radix-4/2 NTT and INTT, the reconfigurable mixed-radix BU operates with all 4 PEs during three stages, while in the remaining stage, only PE0 and PE2 function as a double parallel radix-2 BU. Besides, with four ModMult units integrated, our UniPAM can perform the CWM operation of ML-KEM in a single clock cycle (CC), eliminating the need for an additional BRAM to store intermediate data during the CWM process, which was a limitation found in architectures with only two radix-2 BUs in previous works. Notably, the value of TFs in INTT mode is precalculated to incorporate the factors 2^{-1} or 2^{-2}, as outlined in Algorithm 6, saving 5 modular divisions-by-2 compared to the radix-4 architecture in [29]. Employing complicated mixed-radix method also mitigates information leakage from SCAs that were prevalent in traditional radix-2 methods, as discussed in [27], [28].

With four multiplications of CWM, our UniPAM can continuously process two coefficients per clock cycle, while other operation modes handle four coefficients per CC. By integrating Co/Deco algorithms within the UniPAM, we significantly reduce hardware resource consumption compared to designs relying on multiple independent submodules. Co/Deco algorithms are used to reduce the size of ciphertext in ML-KEM protocol, and their functions are detailed in its specification [2]. However, implementing these algorithms in hardware necessitates modifications to adapt them for efficient operation. Previous works have not described these algorithms in detail for hardware implementation. Notably,

TABLE 1. Interconnections inside the proposed arithmetic unit.

	Input	$mode_{AU}$				
		$NTT_{4/2}$	$INTT_{4/2}$	CWM	+/-	Co/Deco
CE0	A_0	X_0/X_0	U_3/X_0	M	X_0/X_0	$-/-$
	B_0	X_2/X_2	U_1/X_1	U_1	Y_0/Y_0	X_0/X_0
	W_0	ω_2/ω	$\omega_2^{-1}/\omega^{-1}$	V_1	$-$	m/q
CE1	A_1	$U_0/-$	$X_2/-$	g_{2i}	X_2/X_2	$-/-$
	B_1	$U_2/-$	$X_3/-$	f_{2i+1}	Y_2/Y_2	$-/-$
	C_0	$-/-$	$-/-$	g_{2i+1}	$-$	$-/-$
	C_1	$-/-$	$-/-$	f_{2i}	$-$	$-/-$
CE2	A_2	X_1/X_1	V_3/X_2	f_{2i}	X_1/X_1	X_1/X_1
	B_2	X_3/X_3	V_1/X_3	g_{2i+1}	Y_1/Y_1	X_2/X_2
	W_1	$\omega_1/1$	$\omega_1^{-1}/2^{-1}$	g_{2i}	$-$	m/q
	W_2	ω_3/ω	$\omega_3^{-1}/\omega^{-1}$	f_{2i+1}	$-$	m/q
CE3	A_3	$V_0/-$	$X_0/-$	U_2	X_3/X_3	$-/-$
	B_3	$V_2/-$	$X_1/-$	V_2	Y_3/Y_3	X_3/X_3
	W_3	$\omega_4^1/-$	$\omega_4^{-1}/-$	ω	$-$	m/q
Output	Z_0	U_1/U_0	U_0/U_0	$-$	U_0/V_0	V_0/V_0
	Z_1	V_1/V_0	U_2/U_2	U_3	U_1/V_1	U_2/U_2
	Z_2	U_3/U_2	V_0/V_0	V_0	U_2/V_2	V_2/V_2
	Z_3	V_3/V_2	V_2/V_2	$-$	U_3/V_3	V_3/V_3

the study in [17] proposed a dedicated Co/Deco module with specific algorithms and architecture, but the design was complex and not hardware-friendly. In this study, we propose efficient algorithms to implement Co/Deco operations, as shown in Algorithm 7. Since the time required to execute Co/Deco does not conflict with PAOs in ML-KEM, integrating it into UniPAM allows for the reuse of DSPs for multiplications, reducing hardware consumption by eliminating the need for a separate module. Operating in a pipelined manner, our UniPAM can handle each polynomial efficiently: 256 CCs for NTT/INTT modes, 128 CCs for MAC operations, and 64 CCs for other operation modes. As the most time-intensive phase in the ML-KEM process, the timing schedules (Figs. 2–4) are carefully structured to enable continuous operation of this module. This setup showcases the design's ability to balance performance and resource optimization as demands of real-world applications.

2) MODULAR MULTIPLICATION (MODMULT)

Inside the PE, ModMult is considered the most complex component, potentially affecting the circuit's frequency. In this study, Barrett reduction, as adopted in [14], struggles with large bit-size adders, while the recursive reduction method proposed in [31], which exploits the relation $2^{12} \equiv 2^9 + 2^8 + 1$, requires multiple adders for nine additions. To overcome these limitations, we observed that the 24-bit result after multiplication could be expressed as the sum of four 12-bit additions, as shown in this equation:

$$a[23{:}0] \pmod q \equiv 2^{20}.a[23{:}20] + 2^{16}.a[19{:}16] + 2^{12}.a[15{:}12] + a[11{:}0] \pmod q$$

Algorithm 7: Proposed Compression Algorithms

1 $\text{Compress}_d : x \longmapsto \lceil (2^d/q) \cdot x \rfloor \bmod 2^d, d \in \{1, 4, 5, 10, 11\}$
2 $t = m \cdot x \quad (m \approx (2^{35}/q) = 10321339)$
3 $x = (t \gg (35 - d)) + t[34 - d]$
4 $\text{Decompress}_d : y \longmapsto \lceil (q/2^d) \cdot y \rfloor$
5 $t = q \cdot y$
6 $y = (t \gg d) \ + t[d - 1]$

The results of the 4-bit modulus operation are precomputed and stored in two 16-to-1 multiplexers (16-MUXs) and one 10-to-1 multiplexer (10-MUX). Following the same method, the final result of the modular multiplication can be efficiently derived in just two clock cycles. By storing the precomputed values of modular operations in memory, our proposed modular reduction enhances resistance to power attacks during Barrett reduction, as discussed in [26].

3) CONTROL UNIT AND READ/WRITE DATA PATTERN

The CU functions as a finite state machine, controlling the operation of the entire UniPAU architecture during its execution. Upon receiving control signals, the CU activates its internal counter, which serves as the basis for generating three separate addresses across three generators: read (*rAddr*), write (*wAddr*) and TFs (*tfAddr*) address generators. The delay between *rAddr* and *wAddr* corresponds to the number of CCs required for pipelined process from reading data from memory, processing it in the AU, and writing it back to memory. This delay is either 12 CCs for NTT/INTT/CWM modes or 6 CCs for ADD/SUB/Co/Deco modes. The *tfAddr* value is controlled to fetch the appropriate TFs from the pre-calculated ROM, corresponding to each process. The operating mode and three addresses are sent to AU and CMI to synchronize the execution of entire UniPAM, ensuring that all coefficients are accurately processed and stored in their respective locations in the Polynomial Memory.

4) CONFLICT-FREE MEMORY INTERFACE AND POLYNOMIAL MEMORY

According to Algorithms 5 and 6, the hardware architecture of UniPAM must ensure the capability to feed and load 4 coefficients per CC in NTT/INTT modes. The PM is configured with 4 independent RAM banks, each corresponding to an 18kb BRAM. To prevent conflicts when accessing the PM, the conflict-free memory mapping scheme proposed for accessing a single polynomial in [32] and [33] is adopted and modified for this specific case as follows:

$$bankIdx = Order \gg \Delta$$
$$bank\_Addr = \sum_{i=0}^{3} Order[\Delta \times (i+1) - 1{:}\Delta \times i] \mod R$$

where R is number of coefficient entering the AU, Δ denotes the depth of the butterfly unit arrangement, which is set to

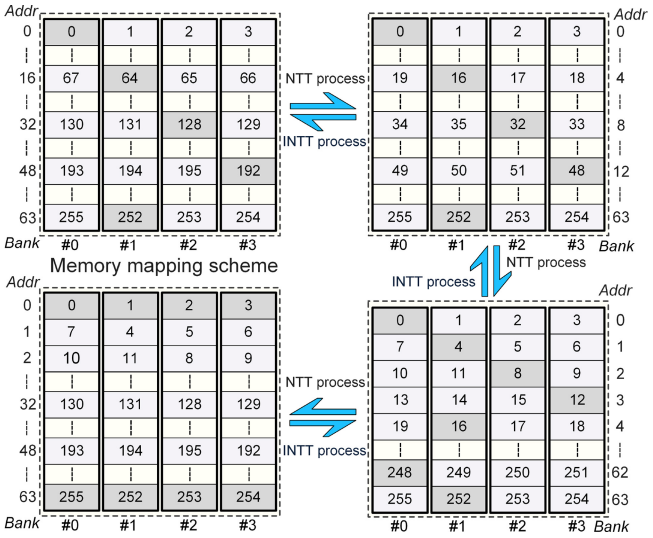

FIGURE 6. Conflict-free Polynomial Memory mapping scheme.

2 in our design, and *Order* refers to the natural order of coefficients in an n-point polynomial.

The CMI transforms initial read and write addresses within the range of n-coefficients into their corresponding locations in memory banks, resulting in pairs of bank addresses (*bank_rAddr*/*wAddr*) and bank indexes (*bankIdx*). Accordingly, the 256 initial coefficients of each polynomial are fixed in a 64×4 layout corresponding to the four memory banks in the PM throughout the process. The bank addresses and data exchanged between the PM and AU are routed to their appropriate memory banks and actual memory locations through the CMI mapping module, following the order of coefficient execution during each operating mode.

The mapping scheme of our polynomial memory is designed to store a single polynomial, as shown in Figure 6, utilizing 64 addresses across 4 banks to accommodate 256 coefficients per polynomial. The positions of these coefficients remain consistent throughout all processes, corresponding with their *bankAddr* and *bankIdx* as defined in the equations above. This arrangement allows for the sampling of 4 coefficients simultaneously across all tasks in our design, maintaining the required order of coefficients during the NTT/INTT processes, which are detailed in Algorithms 5 and 6. The four stages of the NTT/INTT algorithms follow the coefficient order depicted in Figure 6. Accordingly, the CMI manages the addresses of coefficients used during PAOs, sending these addresses to the polynomial memory to retrieve the corresponding coefficient values. Subsequently, the CMI reorders the coefficients to ensure correct alignment between the AU and PM, as illustrated in Figure 1.

C. HASHING AND SAMPLING MODULE

The architecture of the proposed hash module, depicted in Fig. 7, implements a *Keccak-p*[1600, 24] permutation for efficient hash computation and seamless integration with other ML-KEM modules. The 1600-bit state of permutation

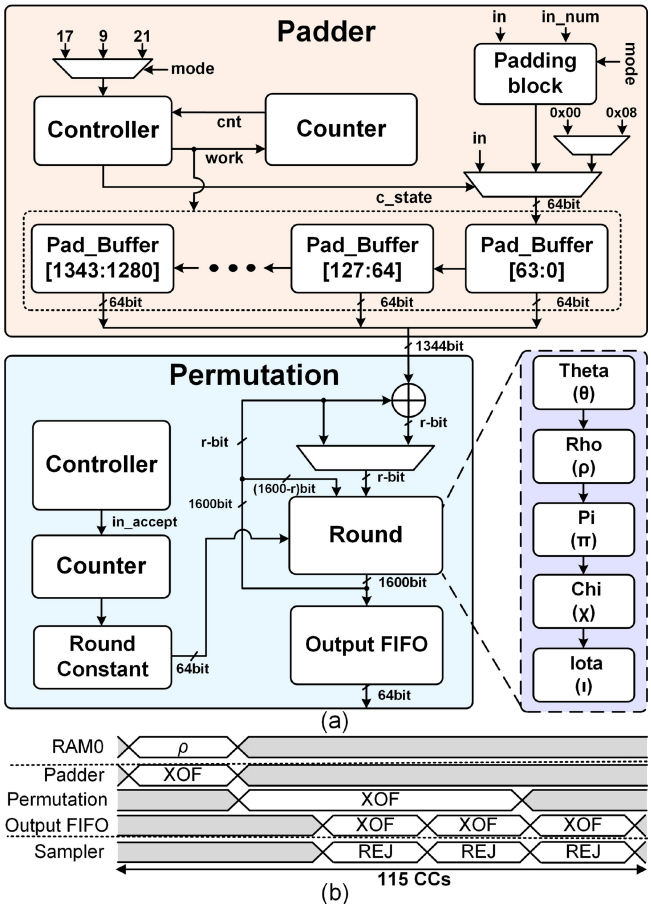

FIGURE 7. (a) Efficient hash architecture and (b) timing diagram.

is stored in a block of 25×64-bit registers, ensuring compatibility with the module's 64-bit data flow. To handle input data, the module performs pre-padding and stores intermediate results in a buffer, enabling efficient and uninterrupted operation. The hash module comprises two submodules, Padder and Permutation, designed to streamline the *Keccak-p* permutation process while meeting the requirements of ML-KEM's hashing and sampling operations.

The Padder module processes a 64-bit input per clock cycle and applies padding based on the hash mode. Padding includes appending a mode-dependent suffix (SHA3: $2'b01$, SHAKE: $4'b1111$) and applying the 10*1 padding scheme. Supported padding sizes include 64 × 9, 64 × 17, and 64 × 21 bits, which correspond to different SHA3 variances used in ML-KEM. The padding process takes 9, 17, or 21 CCs, after which a start signal triggers the Permutation module.

The Permutation module executes the *Keccak-p* [1600, 24] operation, completing each of the 24 rounds of five-step mappings in a single CC. When performing squeeze operations n times, the total processing time scales to $24 \times n$ CCs. Once the permutation is complete, the results are passed to the Output FIFO. After permutation, results are stored in the Output FIFO, ready for subsequent processing. This setup

achieves efficient processing, completing a 24-round *Keccak-p* permutation in 24 CCs, excluding the initial padding time. To ensure reliable operation with minimal latency, additional controllers and buffers are included to coordinate the Padder and Permutation modules.

The hash module plays a critical role in the ML-KEM sampling process. The public matrix $\hat{A}$ is generated using SHAKE128 as an XOF, implemented with a Rejection Sampler. Each *Keccak-p* permutation produces 1344 pseudorandom bits with an acceptance rate of 81.27%, yielding approximately 91 valid samples per iteration. Generating one polynomial of $\hat{A}$ requires at least three permutations and subsequent squeezing operations, following these steps:

Padding: Read the seed ρ from RAM0 and perform padding for 21 CCs using SHAKE128.

Permutation: Execute the 24-round permutation three times, requiring 72 CCs.

Sampling: Use the Rejection Sampler to extract valid samples, which requires 1 CC per sample. Sampling all elements takes an additional 21 CCs, ensuring complete data storage in the output FIFO.

This process is typically completed within an average of 115 CCs, including slack time for handling rejections, ensuring efficient and accurate generation of polynomial matrix $\hat{A}$. To avoid rejection conditions, which can affect the generation time of $\hat{A}$ and lead to inconstant timing, we integrated this process in parallel with the computational tasks in UniPAM to ensure a constant timing schedule in most situations. A similar method is employed for generating the noise polynomials $\mathbf{s}$, $\mathbf{e}$, $\mathbf{y}$, e_1 and e_2, which are sampled using the Pseudorandom Function (PRF_η) and implemented with a Central Binomial Distribution sample. The number of *Keccak-p* permutations required depends on η: two permutations are needed when $\eta = 3$, while only one permutation is necessary for $\eta = 2$. In both cases, the process includes 17 CCs for padding (for a bit-rate $r = 1088$ of SHAKE256) and 24 CCs per *Keccak-p* permutation, resulting in a total of 83 CCs for $\eta = 3$ and 59 CCs for $\eta = 2$, ensuring efficient and flexible sampling across configurations.

IV. IMPLEMENTATION RESULTS AND COMPARISON

A. RESOURCES UTILIZATION AND PERFORMANCE RESULTS

Our proposed ML-KEM architecture was designed and implemented using Verilog HDL. The Register-Transfer Level (RTL) design was synthesized and implemented using the default strategy in AMD Vivado v2022.2 for the AMD Artix-7 (xc7a100tcsg324-3) FPGA platform. For ASIC implementation, the design was synthesized with Synopsys Design Compiler (DC) v2023.12, utilizing Synopsys EDK 14nm technology at 0.8V/125°C. Given the inherent differences in hardware platforms and resource utilization, we employed the ATP and Equivalent Number of Slices (ENS) as primary metrics for efficiency evaluation. The ENS metric calculation, detailed in the table footnotes, accounts for equivalencies between FPGA components such as BRAM, DSPs, and slices. Notably, each slice in the 7-series FPGA comprises 4 LUTs and 8 FFs [34]. Our implementation is configured to support all three security levels and the three main algorithms of ML-KEM (Keygen, Encaps, and Decaps) within a single design, which is flexibly convert the operation modes by simple 2 signal input, that announce and control our design moving to work in three security levels and the three main algorithms of ML-KEM. The maximum operating frequency achieved on the Artix-7 platform is 220 MHz. The resource utilization of the complete design and its key submodules is summarized in Table 2. To assess the efficiency of our implementation compared to other SOTA studies, we provide detailed comparison of execution times and other performance metrics. These comparisons are presented in Table 3 for FPGA-based implementations and Table 4 for ASIC-based implementations.

TABLE 2. Resource consumption of the major modules.

Units	LUT	FF	DSP	BRAM	SLICE
Hash Sampler	6,675	4,440	0	0	2,203
⌊ Hash	6,041	4,204	0	0	2,014
⌊ Sampler	634	236	0	0	189
UniPAM	2,236	983	4	0	826
⌊ PE0	306	102	1	0	151
⌊ ModMult	203	38	1	0	94
⌊ PE1	174	74	0	0	102
⌊ PE2	623	100	2	0	275
⌊ PE3	278	102	1	0	139
⌊ Modular Adder	201	48	0	0	141
Control Unit	1,515	1,124	0	0	328
Encoder/Decoder	427	325	0	0	126
Polynomial Memory	0	0	0	9	0
Total	**12,037**	**6,895**	**4**	**9**	**3,668**

B. COMPARISON AND DISCUSSION WITH RELATED WORKS

Most SOTA studies focus on implementing the CRYSTALS-Kyber scheme efficiently, as summarized in Table 3. While CRYSTALS-Kyber was a pivotal candidate during the NIST standardization process, it is now succeeded by the ML-KEM standard, which introduces key enhancements and modifications. Notably, ML-KEM addresses several limitations identified in Kyber, as highlighted in the Appendix of ML-KEM standard [2]. Despite this progression, existing works largely remain tied to the now-outdated Kyber framework. This work stands out by following the new ML-KEM standard rather than Kyber, ensuring alignment with the latest security and performance requirements established by NIST. To the best of our knowledge, this is the first implementation capable of executing all three primary ML-KEM algorithms (KeyGen, Encaps, and Decaps) within a unified architecture, while also dynamically supporting multiple security levels at runtime. Additionally, our implementation carefully considers vulnerabilities to SCAs,which are serious concerns observed in previous studies [25], [26], [27], [28] and

TABLE 3. Comparison on the FPGA implementation results between the proposed design with related works.

Ref.	SL[1]	CCs[2] [k]	Times[3] [μs]	Area				Freq. [MHz]	Device[4]	ENS[5] (Ratio)	ATP[6] (Ratio)
				LUT	FF	DSP	BRAM				
[14]*	1	4.0/7.0/10.0	34.7/60.8/86.9	18,000	5,000	6	15	115	XC7A100	26,733 (x4.130)	17.189 (x15.769)
	3	7.0/10.0/14.0	60.9/86.9/121.8	16,000	6,000	9	16	115			
	5	10.0/14.0/18.0	89.2/124.9/160.7	16,000	6,000	12	17	112			
[15]*	1	2.1/3.3/4.5	9.7/14.8/20.3	9,300	8,200	4	6	220	XC7A200	17,837 (x2.955)	2.905 (x2.665)
	3	2.7/3.9/5.0	12.3/17.7/22.9	10,400	9,500	8	6				
	5	3.6/4.8/6.0	16.3/21.8/27.1	11,500	10,600	8	10.5				
[16]*	1	1.1/1.5/2.1	5.3/7.2/10.1	16,147	13,868	2	0	208	XC7A200	18,498 (x3.065)	1.955 (x1.793)
	3	1.7/2.4/3.0	8.2/11.5/14.4	16,834	13,722	2	0				
	5	2.7/3.4/4.1	13.0/16.3/19.7	17,827	13,979	2	0				
[17]*	1	3.8/5.1/6.7	23.4/30.5/41.3	7,412	4,644	2	3	161	XC7A12	3,221 (x0.533)	1.470 (x1.348)
	3	6.3/7.9/10	39.2/47.6/62.3								
	5	9.4/11.3/13.9	58.2/67.9/86.2								
[18]*	1	3.3/4.5/6.1	17.8/24.3/32.9	5,487	3,426	2	3.5	185	AC701	2,686 (x0.445)	0.975 (x0.894)
	3	5.6/7.1/9.2	30.2/38.3/49.7								
	5	8.5/10.1/12.9	45.9/54.5/69.7								
[19]*	1	2.4/3.0/4.2	12.1/15.0/21.2	7,302	3,184	4	5	200	XC7A100	3,604 (x0.597)	0.896 (x0.822)
	3	4.2/5.0/7.0	21.2/25.2/34.9								
	5	6.8/8.0/10.2	34.1/40.9/51.1								
[20]*	1	2.9/3.9/5.2	14.3/21.8/30.4	6,910	3,270	2	3	200	XC7A100	2,924 (x0.484)	0.888 (x0.814)
	3	5.0/6.3/7.8	25.4/30.8/38.5								
	5	8.0/9.0/11.2	40.1/44.4/58.0								
[21]**	1	1.8/1.9/3.2	8.5/9.0/15.2	25,757	18,629	21	14	210	Artix-7	13,611 (x2.255)	1.892 (x1.735)
	3	2.6/2.7/4.1	12.3/12.8/20.9								
	5	3.4/3.5/5.8	16.1/16.6/27.6								
TW**	**1**	**1.8/2.7/4.0**	**8.1/12.1/18.0**	**12,037**	**6,895**	**4**	**9**	**220**	**XC7A100**	**6,035 (1.0)**	**1.090 (1.0)**
	3	**3.1/4.1/5.8**	**13.9/18.4/26.1**								
	5	**4.6/6.1/8.0**	**20.7/27.4/36.0**								

[1] SL: Security Level. [2] CCs: Clock Cycles. [3] Function times(KeyGen/Encaps/Decaps) [4] All devices are in Artix-7 family

[5] Equivalent number of slices (ENS) = DSP × 100 + BRAM × 196 + Slice. In which, #Slice = LUT/4 + FF/8 [34]

[6] Area-time product (ATP) = ENS × Total Time (KeyGen + Encaps + Decaps) and ratio between this work (TW) and others.

* Architecture for CRYSTALS-Kyber. ** Architecture for ML-KEM.

TABLE 4. Comparison on the ASIC implementation results between the proposed design with related works.

Works	Tech	Logic Gates (kGE)	Memory (KB)	Frequency (MHz)	Clock Cycles [k]			Total Time (μs)	Total Energy (μJ)	A×T[1](Ratio)
					Keygen	Encaps	Decaps			
TCAS-I'20 [35][2]*	28nm	942+37[5]	12	300	39.6	81.5	136.4	859.0	19.57	237.2[6](x169.4)
TCAS-I'22 [36][3]*	28nm	581+116[5]	24.7	540	18.5	33.7	77.5	206.0	16.24	71.8[6](x51.3)
ESSCIRC'22 [37][3]**	28nm	123	31	35-190	-	-	-	437.6	5.22	26.9(x19.2)
ISSCC'22 [38][4]**	28nm	1,900	448	500	-	-	-	20.8	3.4	6.5(x4.64)
This Work*	**14nm**	**23**	**17.4**	**300**	**4.6**	**6.1**	**8.0**	**62.3**	**2.28**	**1.4**[5](1.0)

[1] ASIC area and time production (AT) = Logic Gate (kGE) × Total Time (ms) / number of PQC algorithm supported.

[2] Support NewHope, Kyber, LAC. [3] Support Kyber, Dilithium. [4] Support Kyber, Saber, NTRU, Dilithium, McEliece, Rainbow.

[5] RISC-V core resource. [6] Result of security level 5. * Support all security level. ** Support only security level 1.

typically ignored in earlier implementations [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]. This comprehensive approach marks a significant departure from earlier works, which either followed the Kyber standard or focused on partial functionalities without incorporating SCA protections.

For example, prior works [16], [17] implemented independent client/server designs. In these implementations, the client-side only supported the Encaps function, while the server-side handled the remaining tasks. However, in the comparison tables of other SOTA studies [18], [19], [20], [21], only the server-side implementation of [16], [17] is presented, overlooking the incomplete nature of these designs. In contrast, our work supports all three ML-KEM functions in a single architecture, addressing the practical requirement for role interchangeability between clients and servers in real-world applications. Furthermore, except for

the study in [21], none of the related works [18], [19], [20] claim to support all three ML-KEM functions or provide independent client/server architectures. Additionally, prior works, including [21], which follows the draft ML-KEM standard, lack the optimization strategies proposed in our design, such as efficient task scheduling and parallelism to minimize latency.

FPGA implementations in [14], [15] are designed for fixed security levels, requiring reinstallation to switch levels, it is an inconvenience for end-users. These implementations consume more hardware resources despite supporting only individual security levels. In contrast, our unified architecture seamlessly supports all ML-KEM security levels while optimizing resource utilization. High-performance designs, such as the fully-pipelined radix-2 NTT architecture proposed by Z. Ni et al. [16], achieved reduced execution times by incorporating multiple butterfly units. However, their approach focuses on Kyber and follows a client/server separation model for each security level, making it less versatile than our comprehensive ML-KEM implementation.

Recent works [18], [19], [20] build upon the architecture in [17], claiming support for three Kyber security levels. However, these studies do not address whether their designs support all three Kyber functions or client/server interchangeability. The work in [18] uses the same NTT architecture as [17], featuring double radix-2 butterfly units to improve data flow. This improvement results in a more compact and faster implementation compared to [17]. However, despite its compactness, the execution time remains high, making it a bottleneck for a public-key cryptosystem. Similarly, the work in [20] by Guo and Li employs a split-radix method to enhance the performance of their earlier design in [17]. While this approach reduces execution time by approximately 20%, it is still considered a low-performance design. When compared to our implementation, the execution time of [20] is approximately 30% slower, highlighting its limitations for high-performance applications.

More recently, Kim et al. [21] proposed a configurable architecture aligned with draft version of ML-KEM standard, supporting 3 security levels similar to our design. However, this work lacks optimized operation scheduling, leading to inefficient utilization of submodules. This inefficiency results in their architecture consuming more than 2× the hardware resources required by our design. Moreover, the ATP metric underscores our design's superiority, demonstrating that our hardware implementation is 1.7× more efficient while maintaining both high performance and resource efficiency.

Table 4 presents a comparison of the ASIC implementations of our work with other state-of-the-art (SOTA) studies [35], [36], [37], [38]. Similar to the FPGA implementation, our ASIC design supports all three primary ML-KEM functions across three security levels. However, to emphasize the superiority of our approach, the comparison focuses on the execution time for the highest security level of ML-KEM. The parameters highlighted in the comparison table clearly demonstrate that our design outperforms the others across several critical metrics: logic gate count, total execution time, and energy consumption. Notably, our architecture achieves this efficiency by leveraging optimized operation scheduling, resource-efficient submodules, and a design approach tailored to the finalized ML-KEM standard, setting it apart from prior implementations. This comprehensive optimization ensures our work achieves superior performance while maintaining lower hardware resource consumption, making it a benchmark for ML-KEM ASIC implementations.

V. CONCLUSION

In this paper, we present a highly efficient hardware implementation of the ML-KEM scheme on the Artix-7 FPGA platform and Synopsys 14 nm ASIC technology. The proposed architecture supports all primary ML-KEM algorithms across three NIST-defined security levels. Our implementation enables the complete execution of ML-KEM tasks independently or in conjunction with other software/hardware platforms, facilitating accelerated performance in specific processes. This work marks the first implementation aligned with the official NIST standard, distinguishing it from prior designs based on Kyber submission. Implementation results demonstrate competitive performance in both execution time and hardware resource utilization. Additionally, proposed architecture introduces optimized arithmetic operations and hashing/sampling modules, meticulously designed to align with the characteristics of ML-KEM. These optimized modules are versatile, capable of operating as standalone accelerators to support software platforms by handling the most computationally intensive components. Furthermore, optimized operation scheduling strategy is incorporated, significantly enhancing the scheme's overall efficiency and mitigating vulnerabilities SCAs aimed at recovering secret key. In summary, our proposed architecture exhibits robust adaptability for diverse KEM applications, providing resilience against both quantum and classical attacks.

REFERENCES

[1] (NIST, Gaithersburg, MD, USA). *Status Report on the Third Round of the NIST-PQC Standardization Process*. Accessed: Jul. 2022. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

[2] (NIST, Gaithersburg, MD, USA). *FIPS 203—Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Accessed: Aug. 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[3] (NIST, Gaithersburg, MD, USA). *FIPS 202—SHA3 Standard: Permutation-Based Hash and Extendable-Out Function*. Accessed: Aug. 2015. [Online]. Available: https://csrc.nist.gov/pubs/fips/202/final

[4] C. Paar and J. Pelzl, "SHA-3 and the hash function Keccak," in *Understanding Cryptography: A Textbook for Students and Practitioners*, vol. 1. Berlin, Germany: Springer, 2010.

[5] T. T. Nguyen, T. T. B. Nguyen, and H. Lee, "An analysis of hardware design of MLWE-based public-key encryption and key-establishment algorithms," *Electronics*, vol. 11, no. 6, 891, 2022.

[6] T. T. Nguyen, S. Kim, Y. Eom, and H. Lee, "Area-time efficient hardware architecture for CRYSTALS-Kyber," *Appl. Sci.*, vol. 12, no. 11, p. 5305, 2022.

[7] U. Banerjee et al., "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2019, no. 4, pp. 17–61, Feb. 2019.

[8] Y. Huang, M. Huang, Z. Lei, and J. Wu, "A pure hardware implementation of CRYSTALS-KYBER PQC algorithm through resource reuse," *IEICE Electron. Exp.*, vol. 17, no. 17, Sep. 2020, Art. no. 20200234.
[9] S. Zhou et al., "Preprocess-then-NTT technique and its applications to Kyber and NewHope," in *Proc. Int. Conf. Inf. Secur. Cryptol.*, Dec. 2018, pp. 117–137.
[10] J. Mu et al., "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof, and implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuit Syst.*, vol. 42, no. 5, pp. 1504–1517, May 2023.
[11] Y. Zhao, X. Liu, Y. Hu, and H. Xiao, "Design of an efficient NTT/INTT architecture with low-complex memory mapping scheme," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 400–404, Jan. 2024.
[12] G. Li, D. Chen, G. Mao, W. Dai, A. I. Sanka, and R. C. C. Cheung, "Algorithm-hardware co-design of split-radix discrete galois transformation for KyberKEM," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 4, pp. 824–838, Oct.–Dec. 2023.
[13] M. Bisheh-Niasar and R. Azarderakhsh, "High-speed NTT-based polynomial multiplication accelerator for CRYSTALS-Kyber post-quantum cryptography," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2021/563, 2021.
[14] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "Instruction-Set Accelerated Implementation of CRYSTALS-Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.
[15] V. B. Dang, K. Mohajerani, and K. Gaj, "High-speed hardware architectures and FPGA benchmarking of CRYSTALS-Kyber, NTRU, and saber," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 306–320, Feb. 2023.
[16] Z. Ni, A. Khalid, D.-E.-S. Kundi, M. O'Neill, and W. Liu, "HPKA: A high-performance CRYSTALS-Kyber accelerator exploring efficient pipelining," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3340–3353, Dec. 2023.
[17] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Trans. CHES*, vol. 2021, no. 2, pp. 328–356, Feb. 2021.
[18] T. H. Nguyen, D. T. Dam, P. P. Duong, B. Kieu-Do-Nguyen, C. K. Pham, and T. T. Hoang, "Efficient hardware implementation of the lightweight CRYSTALS-Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 610–622, Feb. 2025.
[19] W. Guo and S. Li, "Highly-efficient hardware architecture for CRYSTALS-kyber with a novel conflict-free memory access pattern," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 11, pp. 4505–4515, Nov. 2023.
[20] W. Guo and S. Li, "Split-Radix Based Compact Hardware Architecture for CRYSTALS-Kyber," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 97–108, Jan. 2024.
[21] H. Kim, H. Jung, A. Satriawan, and H. Lee, "A configurable ML-KEM/Kyber key-encapsulation hardware accelerator architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 11, pp. 4678–4682, Nov. 2024.
[22] T. H. Nguyen et al., "An area-time efficient hardware architecture for ML-KEM post-quantum cryptography standard," *IEEE Access*, vol. 13, pp. 103834–103847, 2025.
[23] Q. D. Truong and H. Lee, "Efficient polynomial arithmetic and hash modules for ML-DSA and ML-KEM standards," in *Proc. IEEE Asia Pacific Conf. Circuits Syst.*, Nov. 2024, pp. 776–780.
[24] "Recommendations for key-encapsulation mechanisms," NIST, Gaithersburg, MD, USA, Rep. SP-800-227. Accessed: Jan. 2025. https://csrc.nist.gov/publications
[25] P. C. Kocher, "Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems," in *Proc. Annu. Int. Cryptol. Conf. Adv. Cryptol. (CRYPTO)*, Aug. 1996, pp. 104–113.
[26] B.-Y. Sim, A. Park, and D.-G. Han, "Chosen-ciphertext clustering attack on CRYSTALS-Kyber using the side-channel leakage of Barrett reduction," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21382–21397, Nov. 2022.
[27] H. Ma et al., "Vulnerable PQC against side channel analysis—A case study on Kyber," in *Proc. Asian Hardw. Orient. Secur. Trust Symp.*, 2022, pp. 1–6.
[28] Y. Zhao, S. Pan, Y. Gao, J. He, Y. Zhao, and Y. Jin, "Side channel security oriented evaluation and protection on hardware implementations of Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5025–5035, Dec. 2023.
[29] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 94–126, 2022.
[30] Q. D. Truong, P. N. Duong, and H. Lee, "Efficient low-latency hardware architecture for module-lattice-based digital signature standard," *IEEE Access*, vol. 12, pp. 32395–32407, 2024.
[31] F. Yaman, A. C. Mert, E. Öztürk, and E. Savas, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-Kyber PQC scheme," in *Proc. Design, Autom. Test Eur. Conf. Exhibit.*, 2021, pp. 1020–1025.
[32] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, "Highly efficient architecture of NewHope-NIST on FPGA using low-complexity NTT/INTT," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 2, pp. 49–72, 2020.
[33] Q. D. Truong, P. N. Duong, and H. Lee, "Hybrid number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 7, pp. 2039–2043, Jul. 2025.
[34] *7 Series FPGAs Configurable Logic Block: User Guide, UG474 (v1.8)*, Xilinx, San Jose, CA, USA, Sep. 2016.
[35] G. Xin et al., "VPQC: A domain-specific vector processor for post quantum cryptography based on RISC-V architecture," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 8, pp. 2672–2684, Aug. 2020.
[36] Y. Zhao, R. Xie, G. Xin, and J. Han, "A high-performance domain specific processor with matrix extension of RISC-V for module-LWE applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 7, pp. 2871–2884, Jul. 2022.
[37] B. Kim, J. Park, S. Moon, K. Kang, and J. -Y. Sim, "Configurable energy-efficient lattice-based post-quantum cryptography processor for IoT devices," in *Proc. IEEE 48th Eur. Solid State Circuits Conf.*, 2022, pp. 525–528.
[38] Y. Zhu et al., "A 28nm 48KOPS 3.4μJ/Op agile crypto-processor for post-quantum cryptography on multi-mathematical problems," in *Proc. IEEE Int. Solid-State Circuits Conf.*, Feb. 2022, pp. 514–515.

HAESUNG JUNG (Graduate Student Member, IEEE) received the B.S. degree in information and communication engineering from Inha University, Incheon, South Korea, in 2023, where he is currently pursuing the M.S. degree in electrical and computer engineering. He currently specializes in post-quantum cryptography algorithm and architecture design, and his research interests include VLSI architecture design for cryptography.

QUANG DANG TRUONG (Graduate Student Member, IEEE) received the B.S. degree in electrical and electronic engineering from Le Quy Don Technical University, Vietnam, in 2019, and the M.S degree in electrical and computer engineering from Inha University in 2023, where he is currently pursuing the Ph.D. degree. His research interests include algorithm and VLSI architecture design for postquantum cryptography and homomorphic encryption.

HANHO LEE (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in electrical and computer engineering from the University of Minnesota, Minneapolis, USA, in 1996 and 2000, respectively. In 1999, he was a member of Technical Staff-1 with Lucent Technologies, Bell Labs, Holmdel, NJ, USA. From April 2000 to August 2002, he was a member of Technical Staff with Lucent Technologies (Bell Labs Innovations), Allentown, USA, where he was involved in the design of DSP multiprocessor architecture. From August 2002 to August 2004, he was an Assistant Professor with the Department of Electrical and Computer Engineering, University of Connecticut, USA. He has been a Faculty Member with Inha University, Incheon, South Korea, since September 2004, initially with the Department of Information and Communication Engineering and, since 2025, with the Department of Electrical and Electronic Engineering, where he is currently a Full Professor. He leads the Digital Integrated Systems Lab and is the Director of the Artificial Intelligence System on Chip Research Center, Inha University. He was a Visiting Researcher with the Electronics and Telecommunications Research Institute, South Korea, in 2005. He was a Visiting Scholar with Bell Labs, Alcatel-Lucent, Murray Hill, USA, from 2010 to 2011, and a Visiting Professor with The University of Texas at Dallas, USA, from 2017 to 2018. His research interests include algorithm and VLSI architecture design for postquantum cryptography, homomorphic encryption, artificial intelligence, forward error correction coding, and digital signal processing. He served as the General Chair for ISICAS and the Technical Program Chair for ISCAS and APCCAS. He was the Chair of the IEEE Circuits and Systems for Communications Technical Committee. He was a Board of Governor of the IEEE Circuits and Systems Society (CASS) from 2020 to 2023. He is the Vice President of Technical Activities of the IEEE CASS.